

Artificial Intelligence

Fiona Yan Liu

Department of Computing

The Hong Kong Polytechnic University

Learning Outcomes of Lecture 05

- Understand the definition of machine learning and related concepts
- Understand the characters of clustering problem
- Master k-means clustering algorithm
- Understand the characters of classification
- Understand the design of max margin and its usage in support vector machines
- Know the rationale of non-linear problem solving using kernel mapping

Updated Schedule

Week	Date	Topics	Remarks
1	Jan. 13	Introduction to COMP4431	Questionnaire
2	Jan. 20	Heuristic Search	Quiz 1
3	Jan. 27	Knowledge based Agent	Quiz 2
4	Feb. 3	Hill Climbing, Simulated Annealing and GA	Quiz 3
5	Feb. 10	Constrains and Games	Quiz 4
7	Mar. 3	Machine Learning	Quiz 5
8	Mar. 10	Neural Networks	Quiz 6
9	Mar. 17	Deep Learning	Quiz 7
10	Mar. 24	Bayesian	Quiz 8
11	Mar. 31	Demonstration of group project I	
12	Canceled because of public holiday		
13	Apr. 14	Demonstration of group project II	
Revision	Apr. 21	15:30-17:30 TU101	Quiz 9

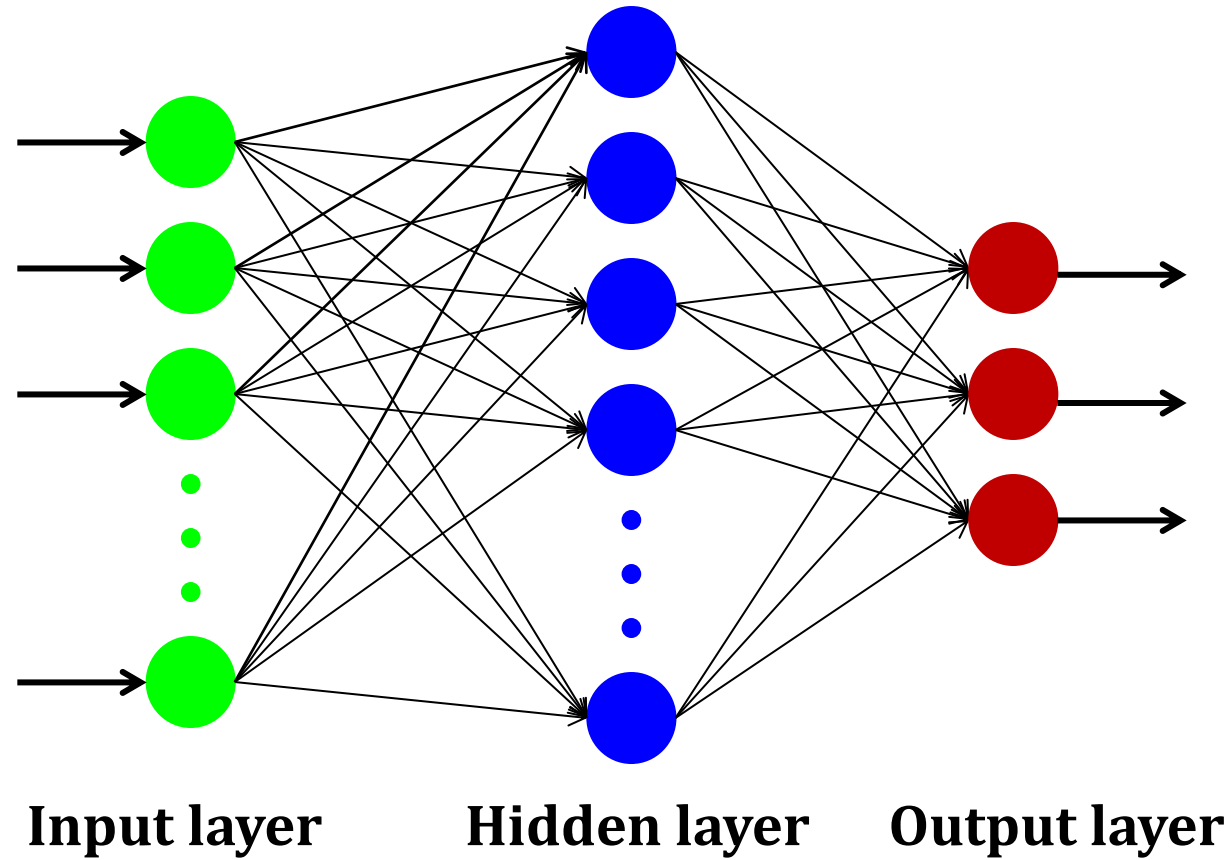
Agenda

- Basic structure
- Artificial neuro
- Perceptron
- Multilayer perceptron

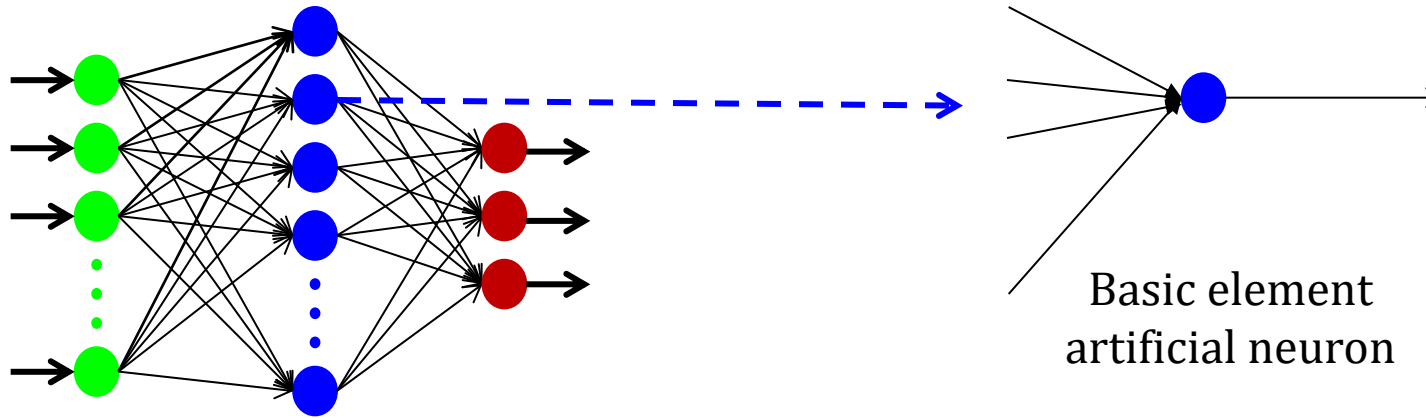
Agenda

- Basic structure
- Artificial neuro
- Perceptron
- Multilayer perceptron

A Typical Artificial Neural Networks



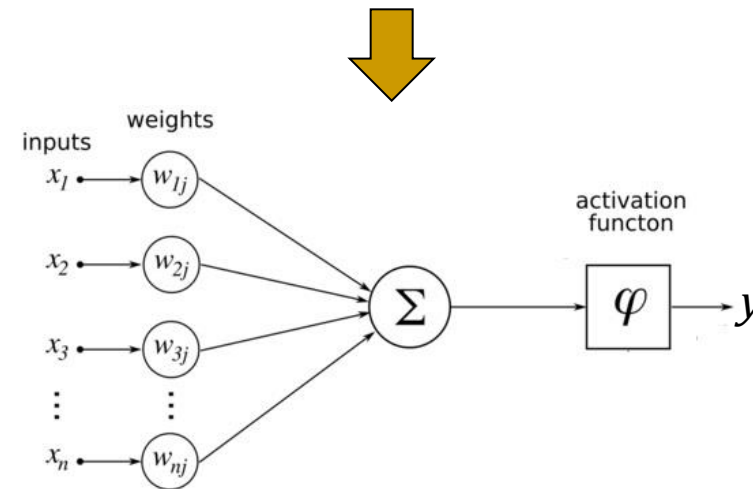
Basic Element



An artificial neuron transforms weighted input nonlinearly:

$$y = \varphi(\mathbf{w}^T \mathbf{x})$$

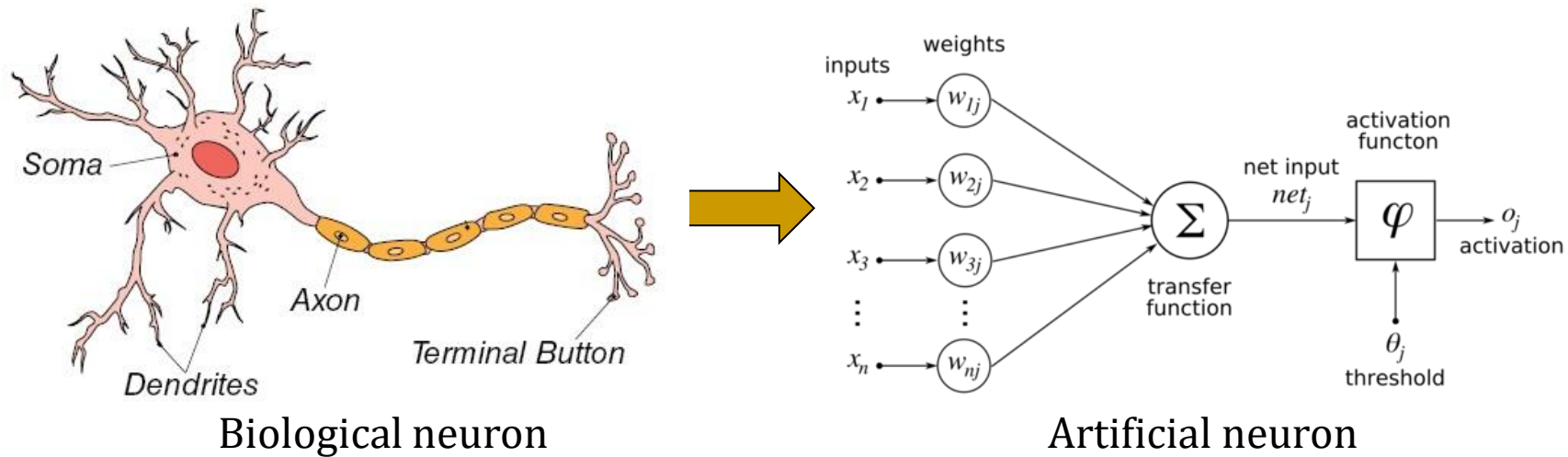
φ is called activation function



Agenda

- Basic structure
- Artificial neuro
- Perceptron
- Multilayer perceptron

Artificial Neuron

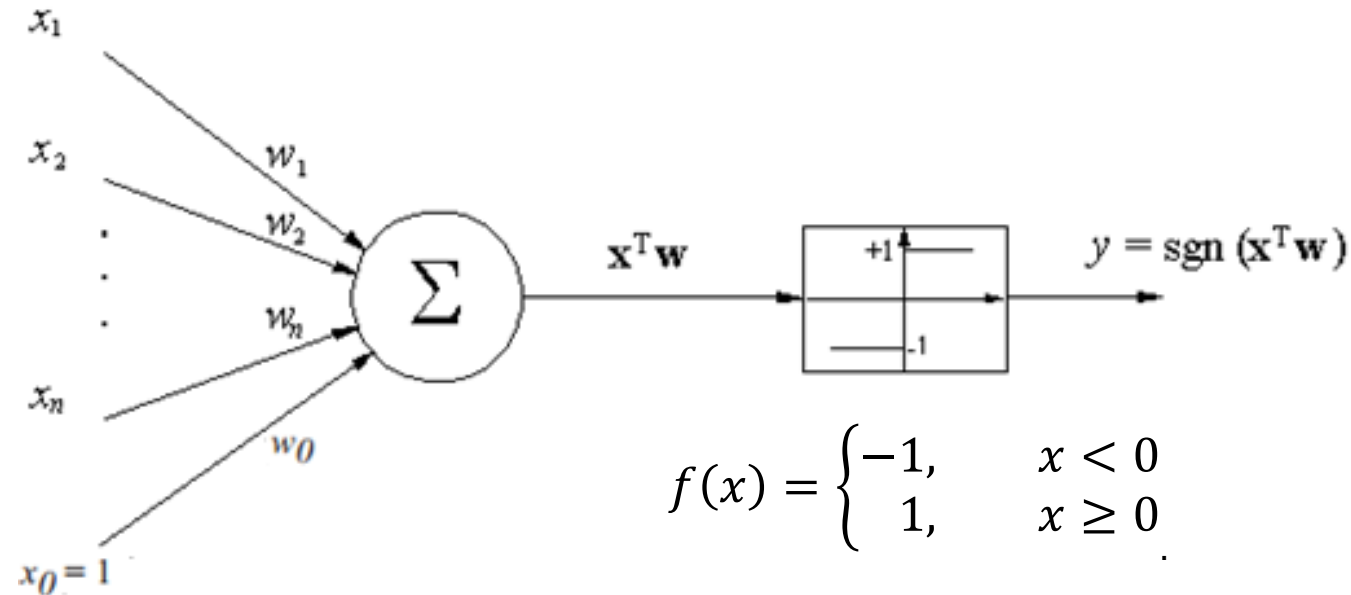


- Why input is a vector?
- Why sum function?
- Why outputs the same value to different inputs?
- Why activation function?
- Why weight?

Agenda

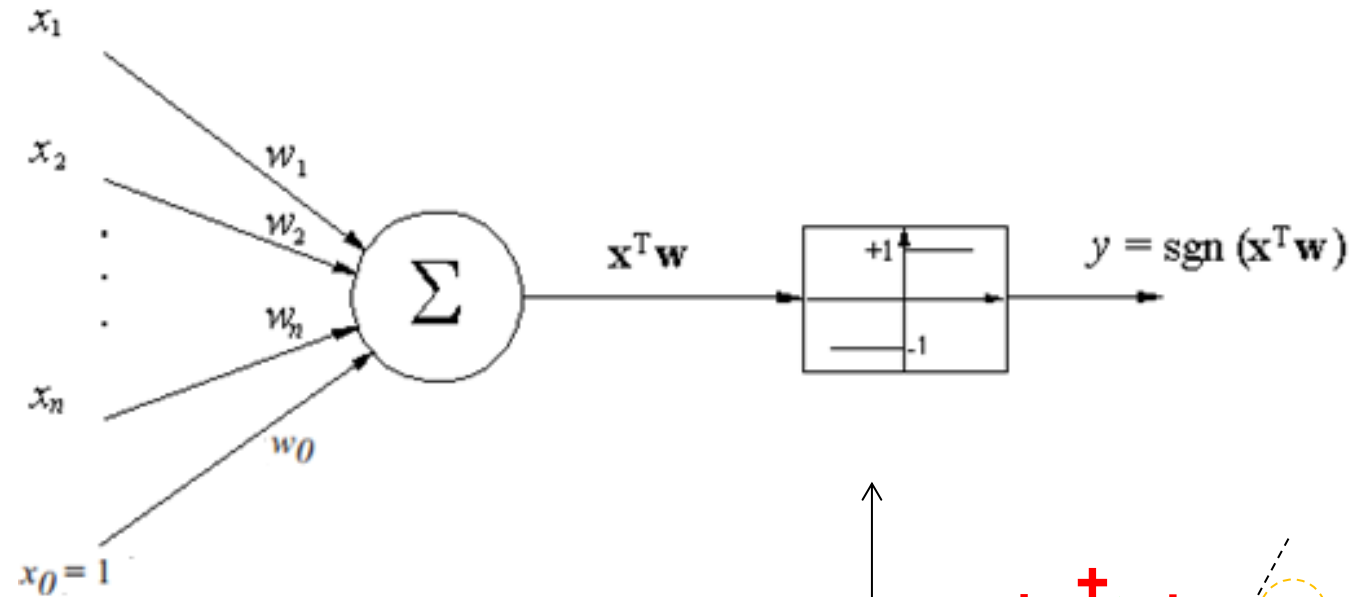
- Basic structure
- Artificial neuro
- Perceptron
- Multilayer perceptron

Perceptron with Activation Function – Sign Function

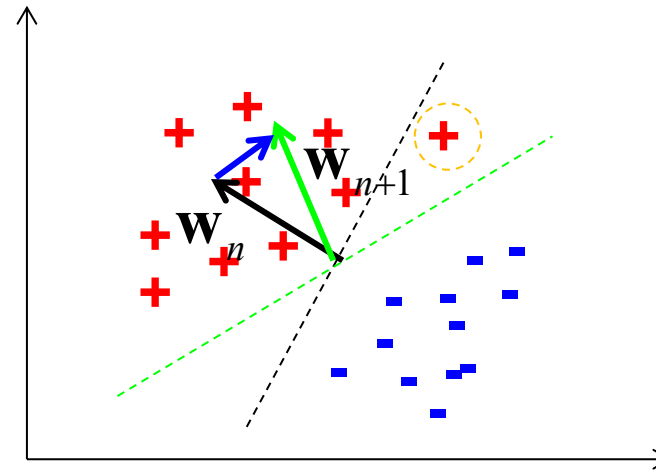


- Input layer and output layer
- Sign function for classification
- Assume $\{\mathbf{x}_d\}_{i=1}^N$ are N -dimensional vector and the labels $y_d \in \{-1, +1\}$
- Perceptron aims to find a weight $\mathbf{w}$ satisfying
$$y_d = \text{sgn}(\mathbf{x}_d^T \mathbf{w})$$

Perceptron Learning Algorithm

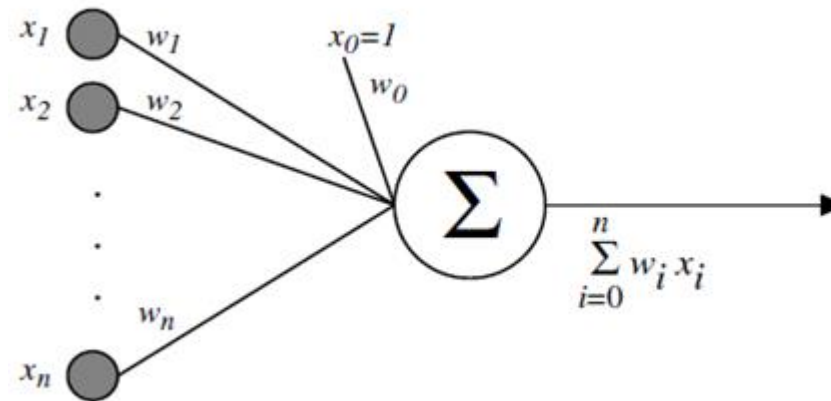


- Initialize random $\mathbf{w}_0$;
- For each $(\mathbf{x}_d, y_d)$, η is the learning rate
 - Calculate the estimated label $\tilde{y}_d = \text{sgn}(\mathbf{x}_d^T \mathbf{w}_k)$
 - Update the weight $\mathbf{w}_{k+1} = \mathbf{w}_k + \eta(y_d - \tilde{y}_d)\mathbf{x}_d$
- The algorithm stops until $\mathbf{w}$ converges.



Perceptron without Activation Function

- Linear regression
- With one-dimensional input
 - $o = w_1x + w_0$
- Loss function
 - $Loss(h_w) = \sum_d (t_d - (w_1x_{1,d} + w_0))^2 / 2D$



Linear Combination with One-dimensional Input

- The partial derivatives of $Loss(h_w)$ with respect to w_0 and w_1 are zero:

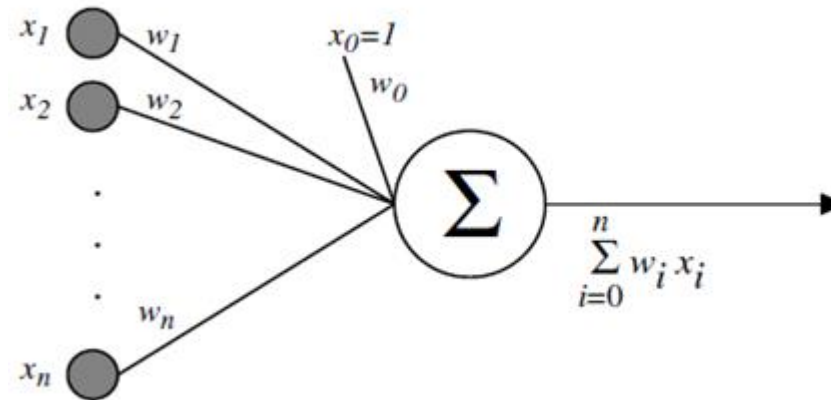
- $\partial \sum_d (t_d - (w_1 x_{1,d} + w_0))^2 / \partial w_0 = 0$

- $\partial \sum_d (t_d - (w_1 x_{1,d} + w_0))^2 / \partial w_1 = 0$

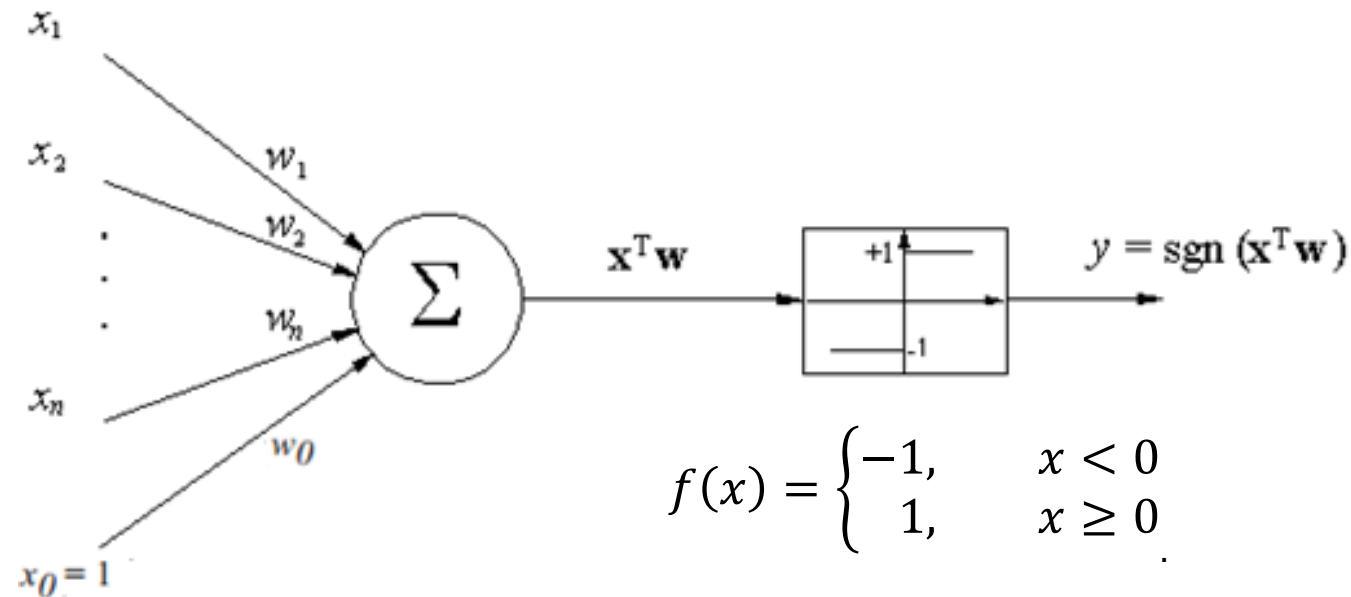
- These equations has analytic solutions

- $w_1 = (D(\sum x_{1,d} t_d) - (\sum x_{1,d})(\sum t_d)) / (D(\sum x_{1,d}^2) - (\sum x_{1,d})^2)$

- $w_0 = (\sum t_d - w_1 (\sum x_{1,d})) / D$



Perceptron with Sign Function



- Nonlinear activation function
$$y_d = \text{sgn}(\mathbf{x}_d^T \mathbf{w})$$
- Only labels $y_d \in \{-1, +1\}$ are known

$$\arg \min_{\mathbf{w}} \frac{1}{2D} \sum \|\mathbf{y}_d - f(\mathbf{w}, \mathbf{x}_d)\|^2$$

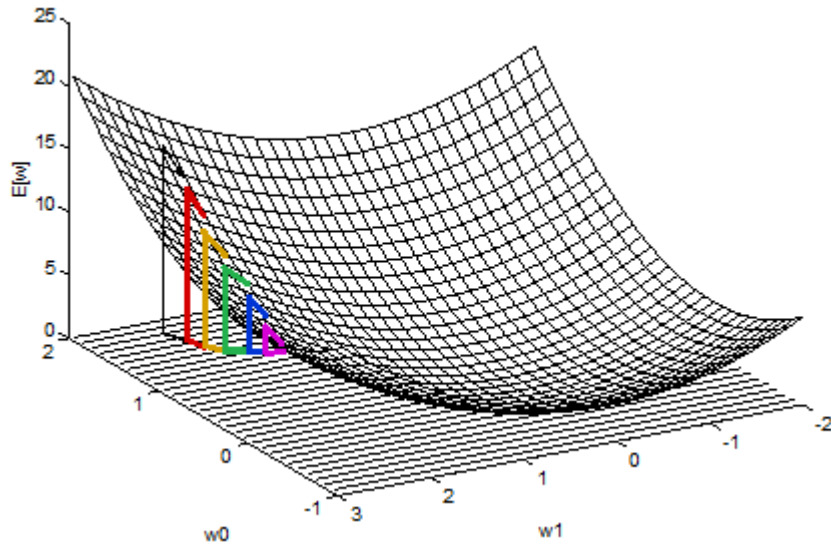
Gradient Descent

- Objective function

$$\arg \min_w \frac{1}{2D} \sum ||t_d - (w_0 + w_1 x_d)||^2$$

- Linear combination with one-dimensional Input

- $o = w_1 x + w_0$



- Gradient

- $\nabla E(\mathbf{w}) = [\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n}]$

- Gradient decent

- $\Delta \mathbf{w} = -\eta \nabla E(\mathbf{w})$

- Training of perceptron with Sgn function

- $\Delta \mathbf{w} = -\eta [\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n}]$

$$= -\eta [\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n}]$$

$$= -\eta \frac{1}{2D} \sum_{d=1}^D ||t_d - (w_0 + w_1 x_d)||^2$$

$$= -\eta \frac{1}{2D} \sum_{d=1}^D 2 * (t_d - o_d) * \frac{\partial (t_d - o_d)}{\partial w_i}$$

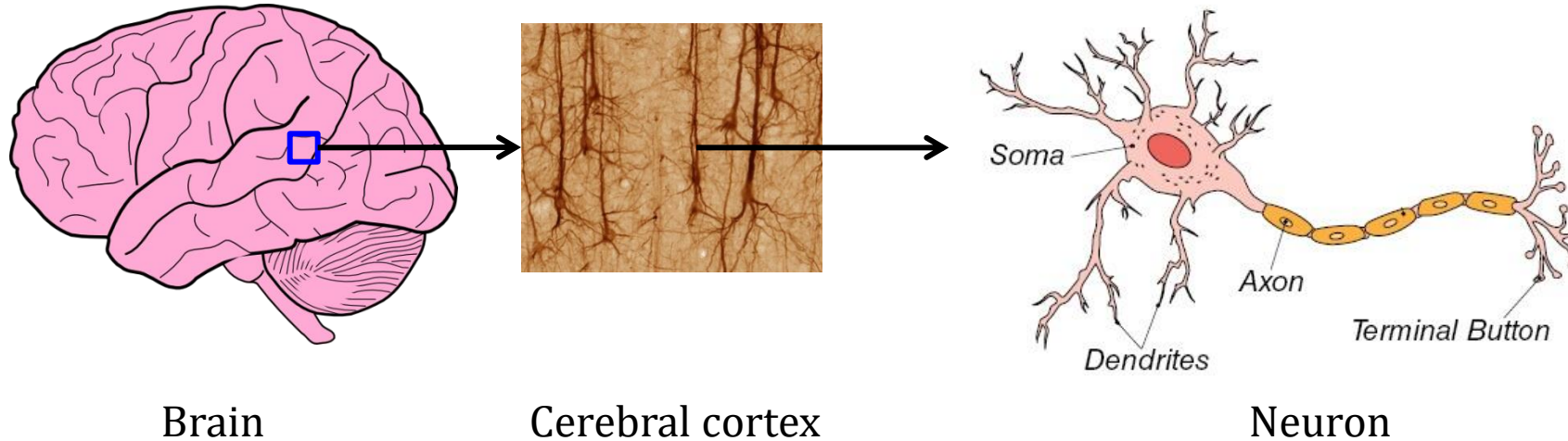
$$= -\eta \frac{1}{D} \sum_{d=1}^D ((t_d - o_d)) * (0 - \frac{\partial (o_d)}{\partial w_i})$$

$$= \eta \frac{1}{D} \sum_{d=1}^D (t_d - o_d) * x_d$$

Agenda

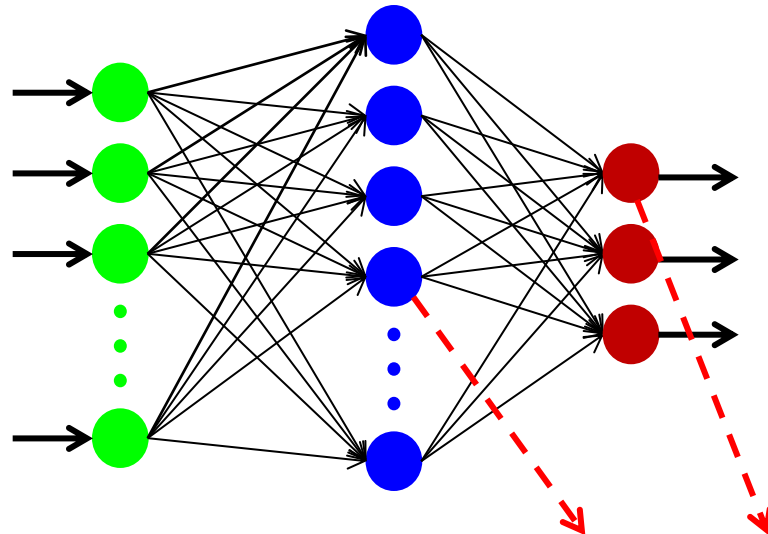
- Basic structure
- Artificial neuro
- Perceptron
- Multilayer perceptron

Difference of Computation Ability

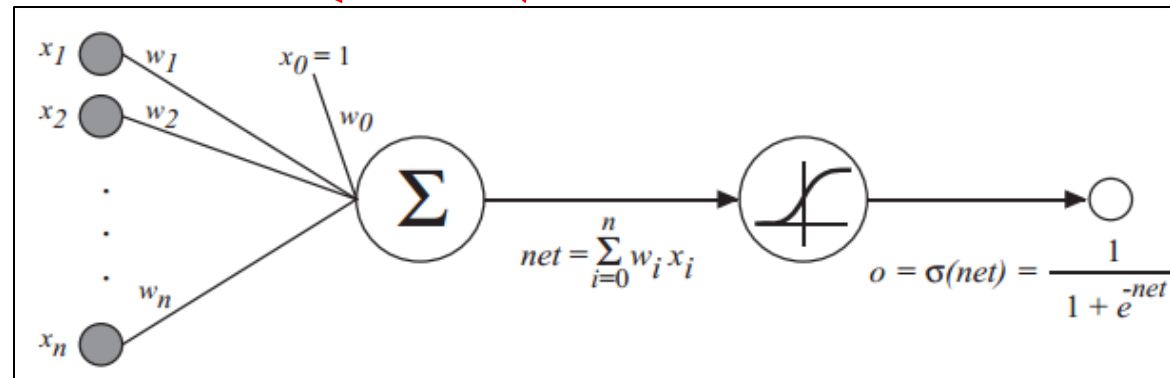


- Each neuron is a cell that uses biochemical reactions to receive, process and transmit information
- Cerebral cortex has the multiple layer structure
- The brain is a collection of about 10 billion interconnected neurons, with approximately 60 trillion connections

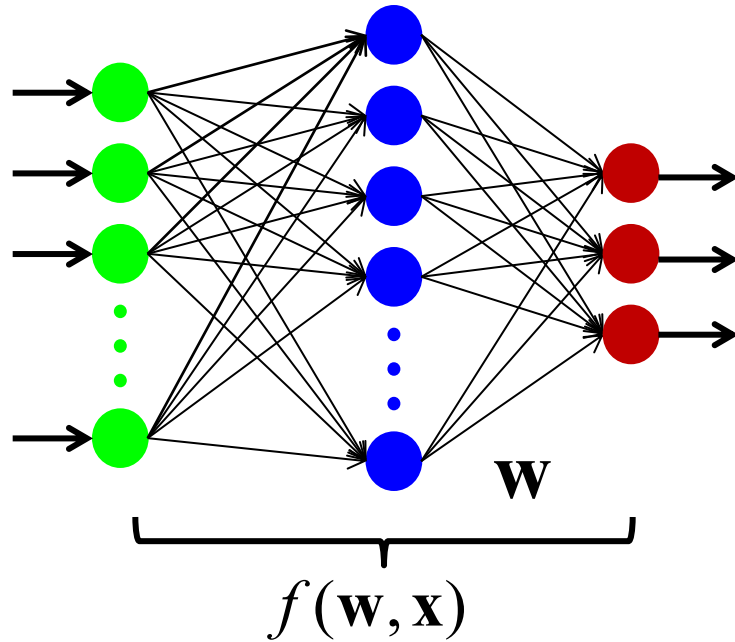
Multilayer Perceptron (MLP)



- One or several hidden layers
- Continuous activation function, sigmoid
- Learning with training data
- Powerful ability for classification and function approximation



MLP Learning: Back Propagation

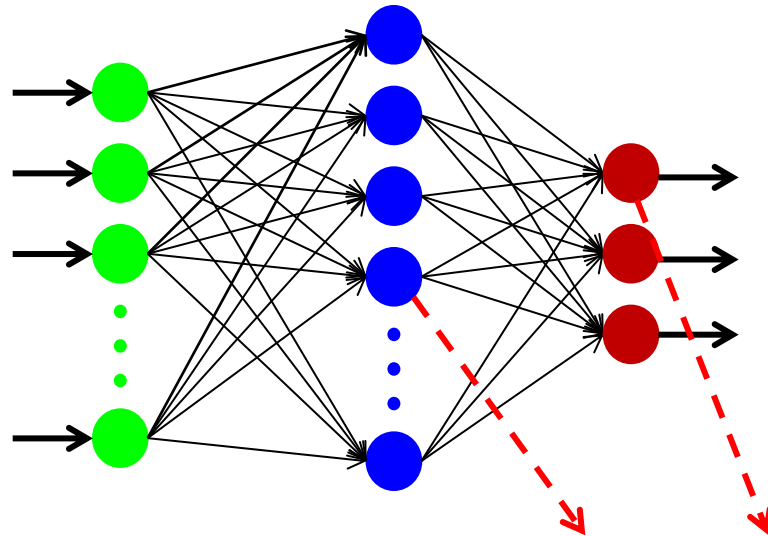


- Back propagation searches weights $\mathbf{w}$
 - to minimize the total error of the network over the set of training examples
 - Gradient descent based method
- Backpropagation consists of two repeated process
 - forward process
 - error propagation process

$\{(\mathbf{x}_d, \mathbf{y}_d)\}_{d=1}^D$ is the training set, back propagation tries to minimize

$$\arg \min_{\mathbf{w}} \frac{1}{2D} \sum ||\mathbf{y}_d - f(\mathbf{w}, \mathbf{x}_d)||^2$$

Forward Processing



- Initialization

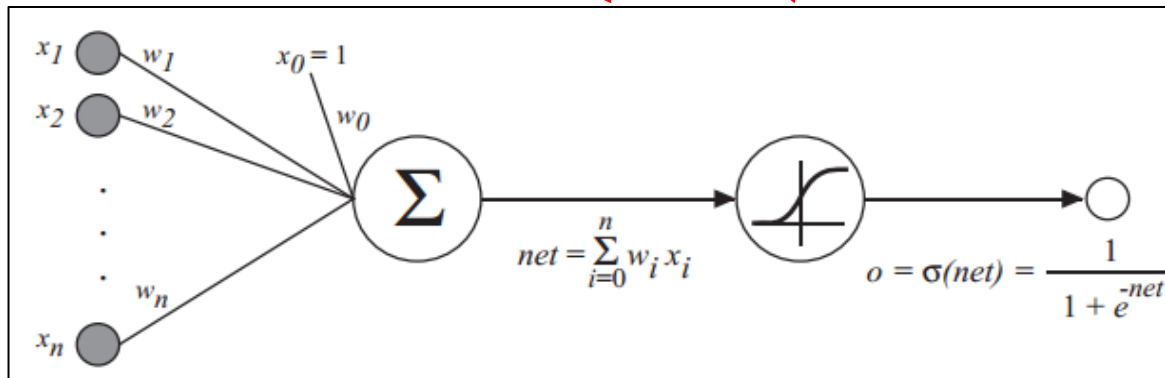
- Assign the weight for each level by random number

- Forward processing

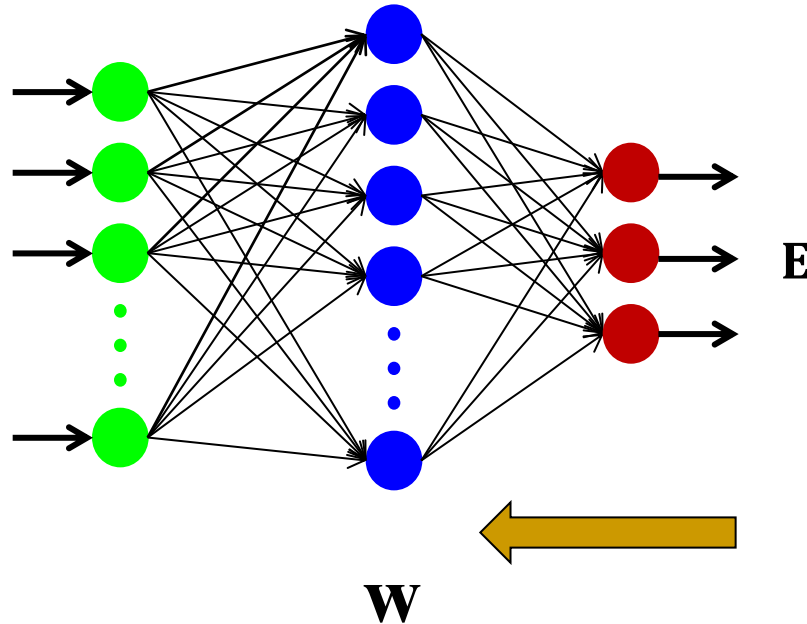
- Calculate the outputs according to the activation function for each layer

- $$o_j = \frac{1}{1+e^{-net}} = \frac{1}{1+e^{-\sum_{i=0}^n w_{ij}x_i}}$$

- $$\widetilde{y}_p = \frac{1}{1+e^{-net}} = \frac{1}{1+e^{-\sum_{j=0}^M w_{jp}o_j}}$$



Error Propagation



$$w_{jp}(k+1) = w_{jp}(k) + \Delta w_{jp}$$

$$w_{ij}(k+1) = w_{ij}(k) + \Delta w_{ij}$$

- For each output neuro p
 $w_{jp}(k+1) = w_{jp}(k) + \eta \delta_p o_j$
Def. $\delta_p = (y_p - \widetilde{y}_p) \widetilde{y}_p * (1 - \widetilde{y}_p)$
- For each hidden neuro j
 $w_{ij}(k+1) = w_{ij}(k) + \eta \delta_j x_i$
Def. $\delta_j = (\sum_p w_{jp} \delta_p) \widetilde{o}_j * (1 - \widetilde{o}_j)$

Gradient Descent for Sigmoid

- For each output neuro p

$$\begin{aligned}\widetilde{y}_p &= \frac{1}{1+e^{-net}} \\ &= \frac{1}{1 + e^{-\sum_{j=0}^M w_{jp} o_j}}\end{aligned}$$

- Gradient decent

$$w_{jp}(k+1) = w_{jp}(k) + \Delta w_{jp}$$

$$\Delta w_{jp} = -\eta \nabla E(w_{jp})$$

- Update the weights for one training data

$$\Delta w_{jp} = \eta (y_p - \widetilde{y}_p) \widetilde{y}_p * (1 - \widetilde{y}_p) * o_j$$

$$Def. \delta_p = (y_p - \widetilde{y}_p) \widetilde{y}_p * (1 - \widetilde{y}_p)$$

$$\Delta w_{jp} = \eta \delta_p o_j$$

$$\frac{\partial E}{\partial w_{jp}} = \frac{\partial}{\partial w_{jp}} \frac{1}{2D} \sum_{d \in D} (y_{p,d} - \widetilde{y}_{p,d})^2$$

$$= \frac{1}{2D} \sum_{d \in D} \frac{\partial}{\partial w_{jp}} (y_{p,d} - \widetilde{y}_{p,d})^2$$

$$= \frac{1}{2D} \sum_{d \in D} 2 * (y_{p,d} - \widetilde{y}_{p,d}) \frac{\partial}{\partial w_{jp}} (y_{p,d} - \widetilde{y}_{p,d})$$

$$= \frac{1}{D} \sum_{d \in D} (y_{p,d} - \widetilde{y}_{p,d}) \frac{\partial}{\partial w_{jp}} (-\widetilde{y}_{p,d})$$

$$= -\frac{1}{D} \sum_{d \in D} (y_{p,d} - \widetilde{y}_{p,d}) \frac{\partial \widetilde{y}_{p,d}}{\partial net_d} * \frac{\partial net_d}{\partial w_{jp}}$$

$$= -\frac{1}{D} \sum_{d \in D} (y_{p,d} - \widetilde{y}_{p,d}) \widetilde{y}_{p,d} * (1 - \widetilde{y}_{p,d}) * o_{j,d}$$